



Be Strong!

UNIVERSITAS LAMPUNG
Berintegritas Anti Korupsi

Pemrograman Berorientasi Objek

Mohamad Zainudin, S.Kom., M.Kom.



S1 Ilmu Komputer PSDKU Way Kanan
Jurusan Ilmu Komputer
Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Lampung

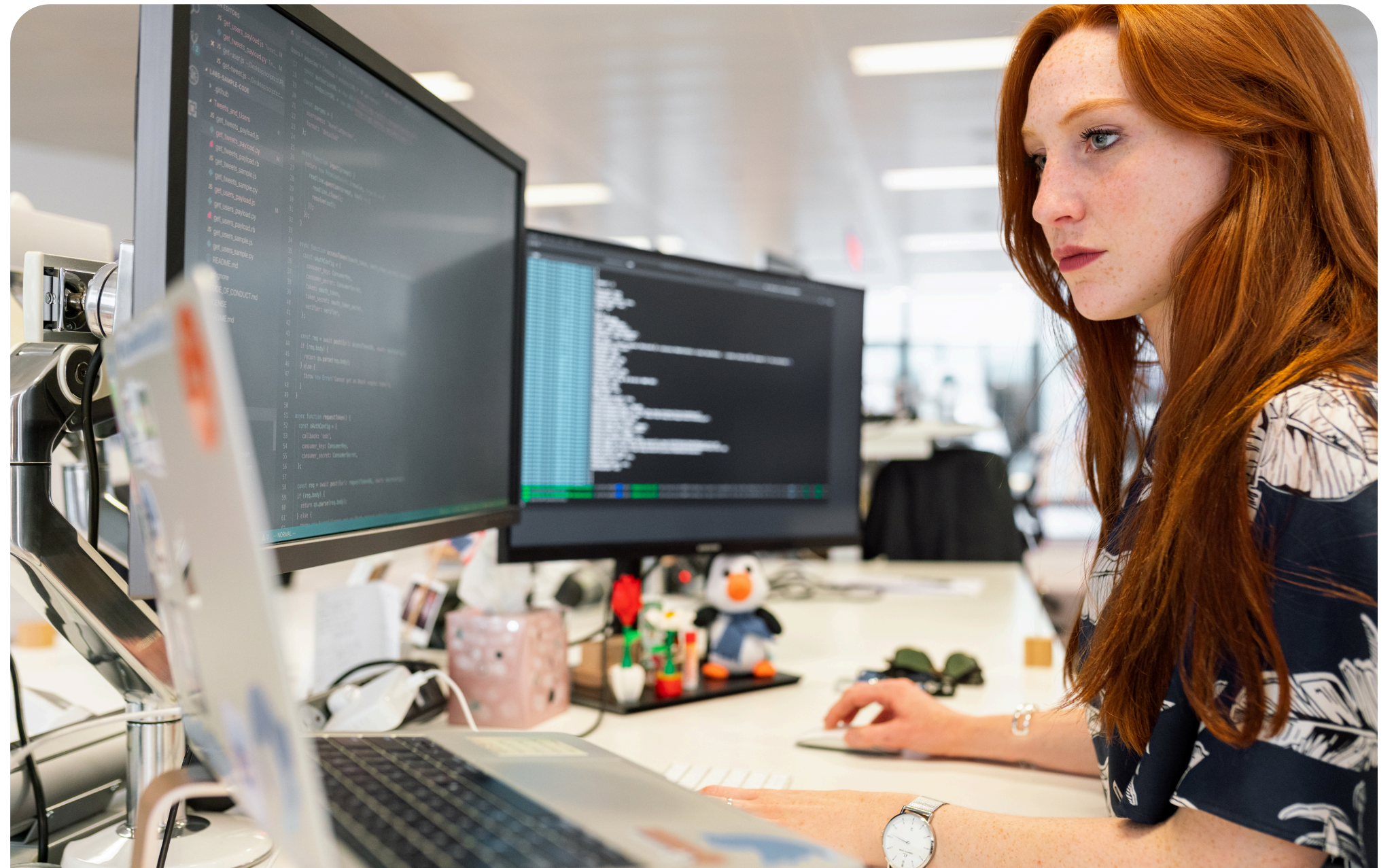
```
out << "Enter row and columns for second matrix." << endl;
int r2, c2;
// Enter elements of first matrix.
out << "Enter elements of matrix 1:" << endl;
for (int i = 0; i < r1; ++i)
    for (int j = 0; j < c1; ++j)
    {
        out << "Enter element a" << i + 1 << " " << j + 1 << " : ";
        cin >> a[i][j];
    }
// Enter elements of second matrix.
out << "Enter elements of matrix 2:" << endl;
for (int i = 0; i < r2; ++i)
    for (int j = 0; j < c2; ++j)
    {
        out << "Enter element b" << i + 1 << " " << j + 1 << " : ";
        cin >> b[i][j];
    }
```

Tata Tertib Kelas

- 01** Kehadiran minimal 80% Teori dan 100% untuk Praktikum (**Keterlambatan 15 Menit**)
- 02** Aktif berpartisipasi dalam diskusi dan deadline tugas wajib dipatuhi
- 03** Mahasiswa kreatif, dan inovatif
- 04** Mahasiswa berpakaian rapih (tidak menggunakan kaos)
- 05** **Melakukan kecurangan = E**

Deskripsi Matakuliah

Pemrograman Berbasis Objek (PBO) merupakan mata kuliah yang membahas konsep, prinsip, dan implementasi paradigma pemrograman berorientasi objek untuk membangun aplikasi yang terstruktur, modular, dapat digunakan kembali, dan mudah dipelihara.



STANDAR KOMPETENSI

ELEMEN KOMPETENSI	KRITERIA UNJUK KERJA
1. Membuat program berorientasi objek dengan memanfaatkan class	1.1 Program dengan menggunakan class dibuat. 1.2 Properti class yang akan direalisasikan dalam bentuk prosedur/fungsi dibuat. 1.3 Data didalam class dibuat mandiri. 1.4 Hak akses dari tipe data (private, protected, public) dikelola.
2. Menggunakan tipe data dan <i>control program</i> pada metode atau operasi dari suatu kelas	2.1 Tipe data diidentifikasi. 2.2 Sintaks program dikuasai sesuai dengan bahasa pemrogramannya. 2.3 <i>Control program</i> dikuasai.
3. Membuat program dengan konsep berbasis objek	3.1 Inheritance pada class diterapkan. 3.2 Polymorphism pada class diterapkan. 3.3 Overloading pada class diterapkan.
4. Membuat program <i>object oriented</i> dengan interface dan paket	4.1 Interface class program dibuat. 4.2 Paket dengan program dibuat.
5. Mengkompilasi Program	5.1 Kesalahan dapat dikoreksi. 5.2 Program bebas salah sintaks dihasilkan.

- Setelah mengikuti mata kuliah Pemrograman Berorientasi Objek, diharapkan mahasiswa mamahami konsep pradigma Pemrograman Berorientasi Objek dengan menggunakan bahasa pemrograman java untuk menyelesaikan masalah.
- Standar SKKNi: J.620100.018.02
Mengimplementasikan Pemrograman Berorientasi Objek

MATERI

Week

Subject

- 1 Kontrak Kuliah, Gambaran Umum OOP, Pengenalan Java & IDE
- 2 Struktur Dasar Java: Tipe Data, Operator, Control Flow, Array
- 3 Konsep Dasar OOP: Class, Object, Field, Method, Constructor
- 4 Encapsulation, Access Modifier, Static, Method Overloading
- 5 Inheritance & Method Overriding
- 6 Polymorphism
- 7 Abstract Class, & Interface

8 UTS

Week

Subject

- 9 Exception Handling & Collection Framework (ArrayList, HashMap)
- 10 Relasi pada OOP: Association, Aggregation, Composition
- 11 File I/O & Object Serialization
- 12 Pengenalan JavaFX (Stage, Scene, Layout, Controls dasar)
- 13 GUI JavaFX dan Validasi Input
- 14 Arsitektur MVC (Model-View-Controller)
- 15 Refactoring Kode, Code Review, & Pengujian

16 UAS



Perangkat Pembelajaran



- Java Development Kit (JDK) : [oracle.com](https://www.oracle.com)
- IDE: Netbeans, Visual Studio Code
- Hackerrank.com, jdoodle
- Oracle Academy Resource

```
}  
render() {  
  return (  
    <React.Fragment>  
      <div className="py-5">  
        <div className="container">  
          <Title name="our" title="product">  
            <div className="row">  
              <ProductConsumer>  
                {(value) => {  
                  console.log(value)  
                }}  
              </ProductConsumer>  
            </div>  
          </div>  
        </div>  
      </React.Fragment>  
    )  
  )  
}
```


Metode Aktivitas Kelas



- Ceramah interaktif dan diskusi



- Praktikum



- Studi kasus dan presentasi



Penilaian

Teori	70%
Praktikum	30%

Tugas	25%
Keaktifan	5%
Kuis	10%
UTS	30%
UAS	30%

Nilai Akhir (0-100)	Huruf Mutu	Angka mutu	Status Penilaian
Program Diploma / Sarjana / Sarjana Terapan / Profesi			
nilai ≥ 76	A	4,0	Lulus
71 nilai $< 75,99$	B+	3,5	Lulus
66 nilai $< 70,99$	B	3,0	Lulus
61 nilai $< 65,99$	C+	2,5	Lulus
56 nilai $< 60,99$	C	2,0	Lulus
50 nilai $< 55,99$	D	1,0	Lulus **
Nilai < 50	E	0,0	Tidak Lulus

**Peraturan Akademik UNILA 2025*



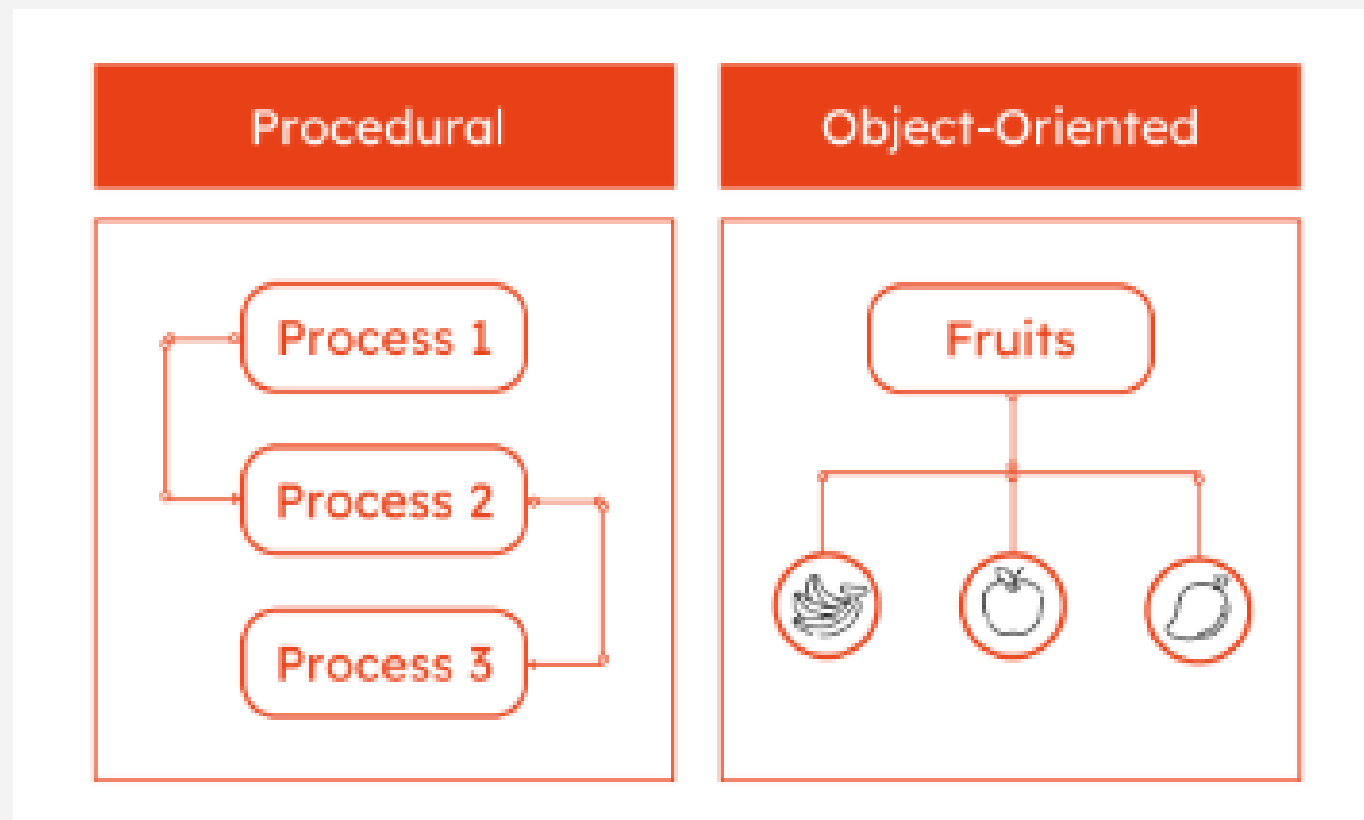
Paradigma Pemrograman

Procedural

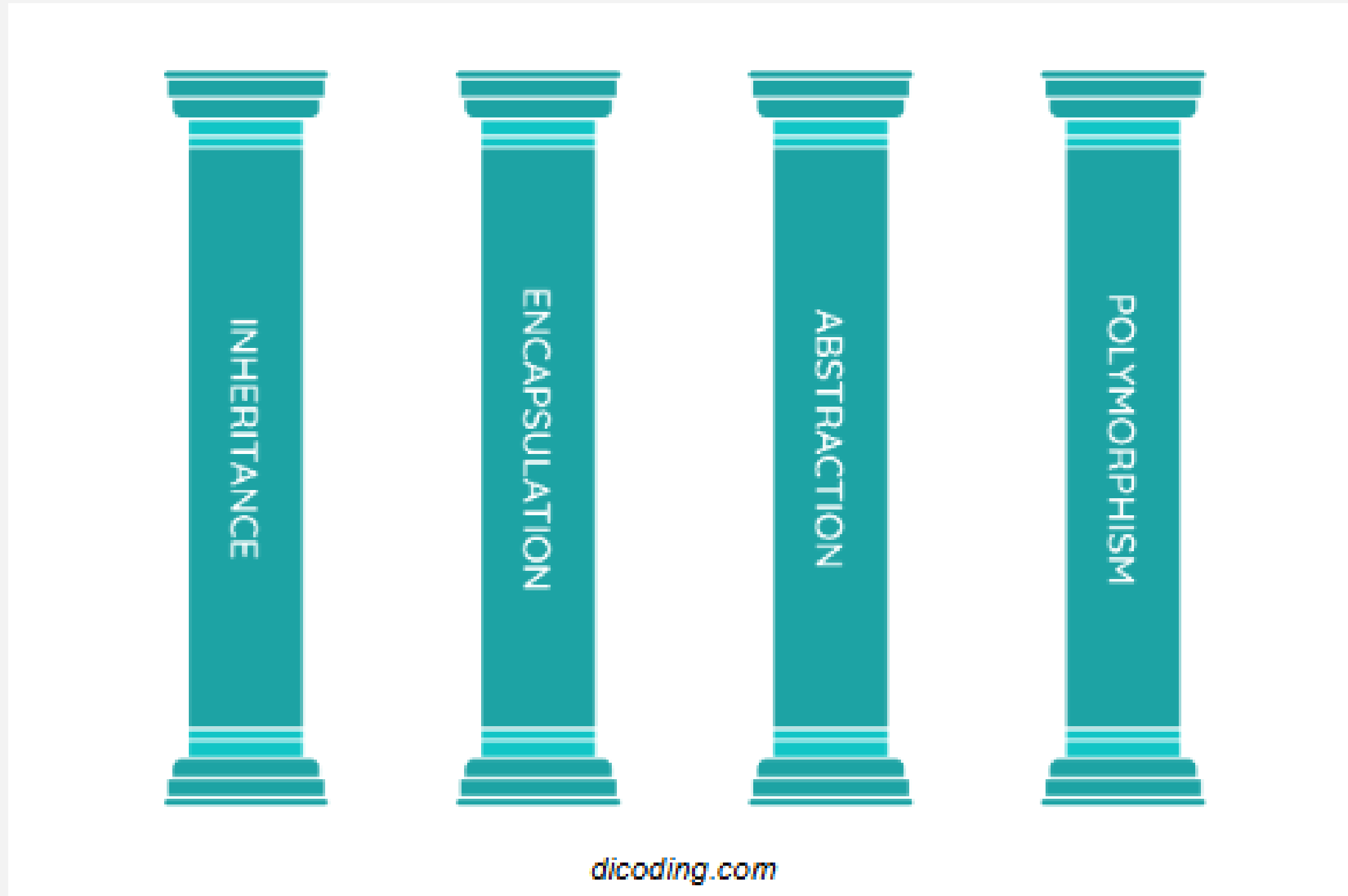
Dalam pendekatan prosedural, program biasanya disusun berdasarkan: prosedur; fungsi; urutan instruksi; data yang diproses oleh fungsi.

Object-Oriented

Program disusun berdasarkan objek yang memiliki data dan perilaku



4 Pillar OOP



KONSEP DASAR

Encapsulation (Enkapsulasi)

memungkinkan data dan fungsi yang berkaitan dikemas dalam satu unit yang disebut class.

Dengan enkapsulasi, informasi sensitif dapat dilindungi, dan akses ke data hanya dapat dilakukan melalui metode tertentu, sehingga mengurangi risiko kesalahan.

Inheritance (Pewarisan)

class anak dapat mewarisi atribut dan metode dari class induk. Ini menghemat waktu pengembangan karena kode yang sama tidak perlu ditulis ulang.

Polymorphism (Polimorfisme)

memungkinkan metode yang sama digunakan pada berbagai class dengan implementasi yang berbeda. Hal ini mendukung fleksibilitas dalam pengembangan sistem.

Abstraction (Abstraksi)

menyembunyikan detail implementasi dan hanya menampilkan fitur yang penting bagi pengguna. Ini membuat sistem lebih mudah digunakan tanpa harus memahami kompleksitasnya.



Encapsulation (Enkapsulasi)

```
class Mahasiswa {  
    private String nama;  
    private int umur;  
  
    public String getNama() {  
        return nama;  
    }  
  
    public void setNama(String nama) {  
        this.nama = nama;  
    }  
  
    public int getUmur() {  
        return umur;  
    }  
  
    public void setUmur(int umur) {  
        if (umur > 0) {  
            this.umur = umur;  
        }  
    }  
}
```

- Definisi: Teknik untuk menyembunyikan detail implementasi suatu objek dan hanya menyediakan akses melalui method khusus (getter/setter).
- Tujuan: Melindungi data agar tidak diakses atau dimodifikasi secara langsung dari luar kelas.
- Analogi: Seperti ATM, kita tidak tahu bagaimana sistem bank memproses uang, yang kita tahu hanya menekan tombol (interface) lalu uang keluar.





Inheritance (Pewarisan)

- Definisi: Mekanisme di mana sebuah kelas dapat mewarisi properti dan method dari kelas lain.
- Tujuan: Mendukung reusability (penggunaan ulang kode) dan memperluas fungsionalitas tanpa harus menulis ulang.
- Analogi: Seperti anak mewarisi sifat dari orang tua, misalnya mata, rambut, atau kebiasaan.





Polymorphism (Polimorfisme)

- Definisi: Kemampuan suatu objek untuk memiliki banyak bentuk (satu interface, banyak implementasi).
- Jenis dalam Java: Compile-time polymorphism (Method Overloading) dan Runtime polymorphism (Method Overriding)





Abstraction (Abstraksi)

- Definisi: Proses menyembunyikan detail implementasi dan hanya menampilkan fitur penting kepada pengguna.
- Tujuan: Memudahkan pemakaian kelas tanpa harus tahu detail teknis di dalamnya.
- Cara di Java: Menggunakan abstract class atau interface.
- Analogi: Saat menyalakan mobil, kita cukup menekan tombol "Start" tanpa perlu tahu detail bagaimana mesin bekerja.





Keunggulan

- Sistem Modular : OOP memecah aplikasi menjadi bagian-bagian kecil yang disebut class dan object
- Scalability : memungkinkan pengembang untuk mengembangkan sistem yang lebih besar tanpa harus membangun dari awal





Be Strong! | **UNIVERSITAS LAMPUNG**
Berintegritas Anti Korupsi



Terima Kasih